



温州肯恩大学
WENZHOU-KEAN UNIVERSITY

2026SPW MGS 3001 WHS01
Python Programming for Business

Text Processing

String Methods, Regex, and LLM-as-Annotator

Dawei Chen

Week 12-1, 12 May 2026

Where We Are

Date	Topic	Note
5/07	Structured Data Cleaning	✓ Completed
5/12	Text Processing	TODAY – String method, regex, LLM-as-annotator
5/14	Research Workshop III	Paper anatomy, section-by-section guidance, writing with AI
Assignment 3 due on 5/15 (Friday) at 23:59		
5/19	Exploratory Data Analysis	Descriptive statistics, correlations, groupby analysis
5/21	Data Visualization	Matplotlib, Seaborn, visualization principle
5/26	Regression: Theory	Interpretation, model specification
5/28	Regression: Application	statsmodels OLS on project data, results presentation
6/02	Research Workshop IV	Draft feedback, presentation prep, consultation
6/04	Final Presentations	8-min pre + 2-min Q&A per project
6/09		
Final report due on 6/10 (Wednesday) at 23:59		

Learning Objectives

Core skill: Transform unstructured text into structured numeric variables that can be used in statistical analysis and regression models.

By the end of this session, you should be able to:

- 1 Use Python built-in and pandas **string methods** to clean and transform text columns
- 2 Write **regular expressions** to **extract patterns** from text data
- 3 Understand **traditional text categorization** methods and how LLMs augment them
- 4 Use **LLM as annotator**: via chatbot, via API, with best practices and validation
- 5 Derive new **numeric variables** from text, categorical variables

Why Text Processing?

Your dataset has rich text fields — but you can't put raw text into a regression.

Raw Text (can't analyze directly)

- **description:** A programming framework for agentic AI
- **topics:** ai, agent, multi-agent
- **full_name:** microsoft/autogen
- **language:** Python
- **license_name:** MIT License



Derived Variables (ready for regression)

- **desc_length:** 39 (word count → proxy for documentation effort)
- **topic_count:** 3 (number of tags → proxy for discoverability)
- **owner_name:** microsoft (extracted from full_name)
- **is_python:** 1 (binary dummy → language == Python)
- **is_permissive:** 1 (MIT/Apache → permissive license flag)

*Today's goal: learn the Python tools to make this **transformation** systematically and reproducibly.*

Each derived variable connects to a **research construct** you can defend in your project:

desc_length → documentation effort, topic_count → discoverability, is_python → ecosystem choice

Python Built-in String Methods

[W3Schools tutorial](#)

Built-in methods that work on **individual strings**. These are the **foundation** — we will apply them at scale with pandas next.

Method	What It Does	Example
<code>s.lower()</code>	Convert to lowercase	"Python" → "python"
<code>s.upper()</code>	Convert to uppercase	"mit" → "MIT"
<code>s.strip()</code>	Remove leading/trailing whitespace	" Python " → "Python"
<code>s.split(',')</code>	Split into a list by delimiter	"ai, agent" → ["ai", " agent"]
<code>s.replace(a,b)</code>	Replace occurrences	"v2.0" → "v2-0"
<code>s.startswith()</code>	Check prefix	"hello".startswith("he") → True
<code>len(s)</code>	Count characters	<code>len("Python")</code> → 6



Key idea: These are standard Python operations on **single strings**.
With pandas `.str`, you can apply them to **entire columns** at once.

Pandas String Methods

[Documentation](#)

When a column contains text (dtype = object), use [.str](#) to access string operations on every row at once.

Method	What It Does	Example
<code>.str.lower()</code>	Convert to lowercase	"Python" → "python"
<code>.str.upper()</code>	Convert to uppercase	"mit" → "MIT"
<code>.str.strip()</code>	Remove leading/trailing whitespace	" Python " → "Python"
<code>.str.contains('x')</code>	Check if string contains pattern	<i>Returns True/False per row</i>
<code>.str.split(',')</code>	Split string into a list	"ai, agent" → ["ai", " agent"]
<code>.str.len()</code>	Count number of characters	"Python" → 6
<code>.str.replace('a','b')</code>	Replace occurrences of a pattern	"v2.0" → "v2-0"
<code>.str.extract(r'(\w+)')</code>	Extract text matching a regex group	<i>Regex-powered extraction</i>



Key idea: `.str` lets you apply Python string methods to an **entire column** without writing a for loop.

String Methods in Action

1. Extract owner name from full_name ("microsoft/autogen" → "microsoft")

```
df['owner_name'] = df['full_name'].str.split('/').str[0]
```

2. Count words in description (proxy for documentation effort)

```
df['desc_word_count'] = df['description'].str.split().str.len()
```

3. Count topics (proxy for discoverability)

```
df['topic_count'] = df['topics'].str.split(',').str.len()
```

Handle empty topics → 0

```
df.loc[df['topics'] == '', 'topic_count'] = 0
```

4. Check if description mentions "AI" or "agent"

```
df['mentions_ai'] = df['description'].str.contains(r'\bai\b|\bagent\b',  
case=False, na=False).astype(int)
```

5. Character length of description

```
df['desc_char_length'] = df['description'].str.len()
```

Regular Expressions (Regex): Introduction

[W3Schools tutorial](#)

A **RegEx** is a sequence of characters that forms a **search pattern**.

Metacharacters

Char	Description	Example
[]	A set of characters	[a-m]
\	Signals special sequence	\d
.	Any character (except newline)	he..o
^	Starts with	^hello
\$	Ends with	planet\$
+	One or more occurrence	he.+o
*	Zero or more occurrence	he.*o
	Either or	cat dog
?	Zero or one occurrence	he.?o

Special Sequences

Seq	Description	Matches
\d	Any digit (0-9)	"v2" → "2"
\w	Word character (letter, digit, _)	"hello_world"
\s	Any whitespace	the space
\b	Word boundary	\bai\b
()	Capture group — extract	(\w+)

 In Python: always prefix regex strings with **r** to avoid escaping issues: `r'\bai\b'`. The **r** means “raw string” — backslashes are treated literally.

Regex in Python: The re Module

[W3Schools tutorial](#)

Python has a built-in package called re for working with Regular Expressions.

Key Functions

Function	Description
re.findall()	Returns a list containing all matches
re.search()	Returns a Match object if match found anywhere
re.split()	Returns list where string is split at each match
re.sub()	Replaces one or many matches with a string

Match Object Properties

Property	Returns
.span()	Tuple with start and end positions of match
.string	The original string passed into the function
.group()	The part of the string where there was a match

```
import re

txt = "The rain in Spain"

# findall — all matches
x = re.findall("ai", txt) # ['ai', 'ai']

# search — first match
x = re.search(r"\bS\w+", txt) # "Spain" at
(12, 17)
```

Using Regex in Pandas

```
# .str.contains() — does the pattern exist? (returns True/False)
df['has_version'] = df['description'].str.contains(r'v\d+\.\d+', na=False).astype(int)

# .str.extract() — pull out a matching group
# Extract owner name from "microsoft/autogen" → "microsoft"
df['owner'] = df['full_name'].str.extract(r'^(\w[\w-]*)')

# .str.findall() — find all matches (returns a list)
df['urls_in_desc'] = df['description'].str.findall(r'https?:\/\/\S+')

# .str.replace() with regex — clean text
# Remove emoji and special characters from descriptions
df['desc_clean'] = df['description'].str.replace(r'[^\w\s]', '', regex=True)
```

From Pattern Matching to Meaning

Regex is powerful for pattern matching, but some text tasks require **understanding meaning**, not just patterns.

Before LLMs, researchers used these approaches to transform text into structured categories.

1 Manual Labeling

Human experts read and classify each text. Gold standard accuracy but slow, expensive, and difficult to scale to large datasets.

2 Dictionary / Lexicon-Based

Predefined word lists (e.g., Loughran-McDonald for finance, LIWC for psychology). Transparent and reproducible but rigid.

3 LDA Topic Modeling

Unsupervised method that discovers latent topics in text. Useful for exploratory analysis but topics may not align with research needs.

4 Fine-Tuned BERT / RoBERTa

Train deep learning models on labeled data. Captures context well but requires training data, compute resources, and technical expertise.

LLM-as-Annotator

The idea:

Send each row's text (e.g., repo description) to an LLM, ask it to classify or score the text, and store the result as a new column. This is "**LLM-as-annotator**" — using AI to construct research variables at scale.

Regex can do

- ✓ Count words, characters, topics
- ✓ Match exact keywords ("AI", "agent")
- ✓ Extract patterns (owner from URL)
- ✓ Replace or remove characters

LLMs can also do

- ★ Classify text by **topic** or **sentiment**
- ★ Summarize long descriptions
- ★ Judge tone, quality, or complexity
- ★ **Categorize without predefined rules**

LLM-as-Annotator combines the strengths: **contextual understanding** like BERT, **no training data needed** like dictionaries, **faster** than manual labeling.

LLM-as-Annotator

A growing body of research supports using LLMs for **text classification and annotation**.

Gilardi, F., Alizadeh, M. and Kubli, M., 2023. ChatGPT outperforms crowd workers for text-annotation tasks. Proceedings of the National Academy of Sciences, 120(30), p.e2305016120.

Zero-shot ChatGPT exceeded crowd-worker accuracy by ~25 percentage points across relevance, stance, topic, and frame detection. Cost: <\$0.003 per annotation.

Törnberg, P. (2023). Chatgpt-4 outperforms experts and crowd workers in annotating political twitter messages with zero-shot learning. arXiv preprint arXiv:2304.06588.

GPT-4 achieved higher accuracy than both expert classifiers and crowd workers for political affiliation classification, with **zero-shot prompting**.

Rathje, S., Mirea, D. M., Sucholutsky, I., Marjeh, R., Robertson, C. E., & Van Bavel, J. J. (2024). GPT is an effective tool for multilingual psychological text analysis. Proceedings of the National Academy of Sciences, 121(34), e2308950121.

GPT performed on par with or better than fine-tuned BERT models for sentiment analysis and psychological construct labeling across 12 languages.

LLM-as-Annotator

A growing body of research supports using LLMs for **text classification and annotation**.

Churchill, A., Pichika, S., Xu, C., & Liu, Y. (2026). GPT models for text annotation: An empirical exploration in public policy research. Policy Studies Journal, 54(1), e70034.

Systematic evaluation of GPT-3.5 and GPT-4 for policy text annotation — recommends **few-shot prompting with validation**.

Mehmood, M. U., Hussain, S., Wang, W. L., & Malik, M. U. (2025). LLM-based Content Classification Approach for GitHub Repositories by the README Files. arXiv preprint arXiv:2507.21899.

Fine-tuned BERT/RoBERTa models achieved F1 = 0.98 for classifying GitHub README sections.

If you use LLM-generated labels in your project, cite proper papers in your Methods section to justify the approach.

LLM-as-Annotator: **Via Chatbot**

You are already familiar with ChatGPT, Claude, or Gemini. Here's how to use them for text annotation.

- 1 Prepare Your Data**
Structure your text data in a CSV or spreadsheet format ready for batch input.
- 2 Write a Clear Prompt**
Provide clear classification instructions, category definitions, and 2-3 few-shot examples.
- 3 Copy-Paste in Batches**
Copy a batch of texts into the chatbot, send your prompt, and collect the classifications.
- 4 Copy Results Back**
Paste the LLM's output back into your dataset. Verify a sample for quality.



Best for: small-to-medium datasets (hundreds of items), exploratory analysis, prompt refinement before switching to API.

LLM-as-Annotator: Via Chatbot

EXAMPLE PROMPT

Classify each GitHub description:

[framework, tool, library, dataset, demo]

Examples:

- "A deep learning framework" → framework
- "Command-line tool" → tool

Classify:

1. A Python library for NLP
2. Multi-agent conversation framework

Return ONLY the label.

LLM-as-Annotator: Via API

For larger datasets, use the API to automate the annotation process programmatically.

```
# Pseudocode: classify repo descriptions using an LLM
import openai  # or anthropic, etc.

def classify_description(text):
    prompt = f"""Classify this GitHub repo description
into one of: [framework, tool, library, dataset, demo].
Return ONLY the category label.
Description: {text}"""
    response = openai.chat(model=..., prompt=prompt)
    return response.strip().lower()

# Apply to every row
df['repo_category'] = df['description'].apply(classify_description)
```

- The `f` means this is an f-string (formatted string literal). It allows variables to be inserted directly using `{}`
- Triple quotes create a multi-line string. So the string can span multiple lines without `\n`.

LLM-as-Annotator: API vs Chatbot

Choose the right approach based on your project needs and technical comfort level.

Dimension	Chatbot (ChatGPT/Claude)	API (Python Code)
Ease of Use	No coding required, familiar web interface	Requires Python coding knowledge
Scalability	Manual copy-paste, limited batch size	Automated, thousands of rows
Cost	Free tier or subscription	Pay per token (~\$0.001-0.01/annotation)
Reproducibility	Hard to reproduce exactly	Fully reproducible with code
Speed	Interactive, human-paced	Batch processing, much faster
Best For	Small datasets, exploration, learning	Large datasets, production pipelines

Recommendation: Start with the chatbot to refine your prompt and validate categories on a small sample. Then switch to the API for your full dataset. This two-step approach saves time and money.

LLM-as-Annotator: Best Practices

- 1 Define Categories A Priori**
Decide your labels before running the LLM, not after. Let theory guide your categories.
- 2 Use Structured Output**
Ask for a single label or number, not free text. This makes downstream analysis much easier.
- 3 Validate a Sample Manually**
Spot-check 10-20% of LLM labels against your own judgment. Report accuracy.
- 4 Document Your Prompt**
The exact prompt is part of your Methods section. Include it in an appendix.
- 5 Report Inter-Rater Reliability**
Compare LLM labels to human labels using Cohen's kappa or accuracy. Report both.
- 6 Set Temperature to 0**
Setting temperature=0 makes LLM output deterministic and reproducible across runs.
- 7 Report Model Version and Date**
Models are updated and outputs may change. Always report which model and when.

From Text to Research Variables

Every variable you construct should map to a research construct you can defend in your paper.

Raw Field	Derived Variable	Type	Research Construct
description	desc_word_count	Numeric	Documentation effort
topics	topic_count	Numeric	Discoverability / categorization
full_name	owner_name	Categorical	Developer identity
language	is_python	Binary (0/1)	Ecosystem indicator
license_name	is_permissive	Binary (0/1)	Commercial openness
description	mentions_ai	Binary (0/1)	AI positioning / framing
owner_type	is_org	Binary (0/1)	Organizational vs. individual
created_at	repo_age_days	Numeric	Market maturity / first-mover

In your paper: the Data & Methods section should explain what each variable measures and why you chose to construct it that way. The "Research Construct" column is what goes in your paper.

Constructing Binary (Dummy) Variables

Binary variables (0 or 1) are essential for regression — they let you include categorical information as predictors.

```
# Method 1: From a categorical column (owner_type)
df['is_org'] = (df['owner_type'] == 'Organization').astype(int)

# Method 2: From text matching (description contains keyword)
df['mentions_ai'] = df['description'].str.contains(
    r'\bai\b|\bartificial intelligence\b', case=False, na=False).astype(int)

# Method 3: From a set of values (permissive license)
permissive = ['MIT License', 'Apache License 2.0', 'BSD 3-Clause']
df['is_permissive'] = df['license_name'].isin(permissive).astype(int)

# Method 4: From a numeric threshold
df['is_popular'] = (df['stargazers_count'] > 10000).astype(int)
```

Variable Types for Regression

Numeric (Continuous)

Examples:

```
desc_word_count  
topic_count  
repo_age_days  
stargazers_count
```

In regression:

Can be used directly as independent or dependent variable in OLS regression.

Binary (Dummy, 0/1)

Examples:

```
is_org  
is_permissive  
is_python  
mentions_ai
```

In regression:

Included directly in regression.
Coefficient = difference in outcome between 0 and 1 group.

Categorical (Multiple levels)

Examples:

```
language  
license_name  
owner_type
```

In regression:

Must be converted to dummies first using `pd.get_dummies()`. One category is dropped (reference).

Lab Exercise

Derive research variables from your cleaned dataset. Use your own data if you have text to label; otherwise use the GitHub dataset (on Canvas).

- 1 Load your cleaned CSV** Use the output from last session's 02_cleaning.ipynb
- 2 Create desc_word_count** `.str.split().str.len()` on description
- 3 Create topic_count** `.str.split(',').str.len()` on topics (handle empty)
- 4 Create is_org** `(owner_type == 'Organization').astype(int)`
- 5 Create binary variable** `is_python`, `is_permissive`, `mentions_ai` — your choice
- 6 Document each variable** What does it measure? Why did you construct it this way?

Save as 03_variable_construction.ipynb — keep it separate from your cleaning notebook.

What's Next



Wednesday (5/14): Academic Writing

- ➔ Paper anatomy, section-by-section guidance — bring a rough outline of your research question and variables

Monday (5/19): Exploratory Data Analysis

- ➔ Descriptive statistics, correlations, groupby — using the variables you construct today

Your task after class

Complete the lab exercise and draft a description of your variables for the Data & Methods section

Q&A

dchen@kean.edu CBPM B214

Office hours: Tue & Thu 11:15–12:30 | Wed 14:00–16:30